

소프트웨어 개발 완료 보고서

생활 맞춤형 식단 및 체중 관리 어플리케이션 / 청춘 일상 다이어트 총력전

문서 버전 v1.0

작성일 2026-09-27

작성 부서 개발자

작성자 정유운



문서 관리

항목	내용
문서명	소프트웨어 개발 완료 보고서
프로젝트명	생활 맞춤형 식단 및 체중 관리 어플리케이션
문서 ID	DOC-G5C-001
문서 버전	V1.0
보고 대상	과목 담당 교수
보고일	2026-10-01
보안 등급	공개

1. 개발 개요

항목	내용
개발 목적	식단 및 체중을 보다 더 간편하고 세부적으로 관리하기 위해 만들어 졌으며 해당 소프트웨어를 통하여 사용자가 현재 건강 상태를 자신의 식습관과 체중을 통해 파악하고 더 나아가 식습관 관리를 통하여 건강 상태를 한 층 끌어올릴 수 있게 만들 수 있도록 한다.
개발 기간	2026-09-15 ~ 2026-09-29
발주/요청 부서	발주자 : [2023114991 오은찬]
개발 주관 부서/업체	금오공과대학교
책임자	개발자 : [20231055 정유운]
참여 인력	발주자 : [2023114991 오은찬] 개발자 (전체) : [20231055 정유운]

2. 개발 범위 및 주요 결과

구분	계획 범위	완료 결과	상태	비고
기능	필수 기능 [REQ-001 ~ REQ-005] 대부분 구현	모두 구현	완료	
화면	입력에 예외가 발생하지 않도록 안전하게 구현	부록 참고	완료	
데이터	Records.json	모두 구현,	완료	

	Foods.json 을 통한 JSON 파일 파싱 및 쓰기	데이터 구조 변경 있음 (부록 참고)		
--	---------------------------------------	----------------------	--	--

3. 주요 개발 내용

번호	개발 항목	주요 내용	관련 산출물
1	GUI_Manager 설계 및 개발	Tkinter 객체에 대한 관리 클래스를 만들어 객체들이 따로 놓지 않고 일관성 있는 형태를 갖추도록 하고 GUI 클래스를 통하여 해당 관리 객체를 생성 및 관리를 한다.	Gui_Manager.py
2	Operation_Manager 개발	GUI_Manager의 관리 객체의 동기화 함수의 값을 주는 역할을 수행하며 관리 객체의 이벤트 함수를 통하여 데이터를 Data_Manager로 전달한다. 데이터의 전달 과정에서 데이터의 가공을 수행한다.	Operation_Manager.py
3	Data_Manager 개발	Operation_Manager에 전달받은 값을 통하여 데이터의 저장 및 삭제를 수행한다.	Data_Manager.py

산출물 링크 : (<https://github.com/Ovie-hex/Software-Project-P1-Group-5>)

4. 요구사항 이행 현황

요구사항 ID	요구사항 내용	구현 결과	검증 결과	상태
REQ-001	식단 기록 및 관리	식단 관리 페이지를 이용하여 해당 식단 기록을 관리함, 또한 음식 관리를 통하여 음식을 추가하거나 삭제할 수 있다.	검수 보고서 참고	충족
REQ-002	체중 기록 및 변화 조회	식단 관리 페이지를 이용하여 해당 체중 기록을 관리함 메인 페이지에서 현재 시간을 기준으로 7일 전 까지의 두 비교군을 골라 체중의 변화를 볼 수 있다.	검수 보고서 참고	충족
REQ-003	일일 섭취 칼로리 계산	식단 관리 페이지 및 메인 페이지에서 식단에 따른 일일 섭취 열량이 표기된다.	검수 보고서 참고	충족
REQ-004	다이어트 총력전 리포트	메인 페이지에서 선택된 날짜의 총 섭취 칼로리와 목표 칼로리를 비교하여 성공, 실패 여부를 가린다.	검수 보고서 참고	충족
REQ-005	음식 검색 기능	음식 관리 페이지에서 검색창에 입력 문자하고 매칭되는 모든	검수 보고서 참고	충족

		음식의 결과가 표시됨		
--	--	-------------	--	--

5. 시험 및 품질 활동 결과

시험 유형	수행 기간	대상/범위	케이스 수	성공	실패	조치 상태
기능 테스트	2026-09-25	요구사항 전체	5건	5건	0건	완료
디바운스 테스트	2026-09-25	GUI 버튼	1건	1건	0건	완료
타입 오류 테스트	2026-09-25 ~ 2026-09-27	GUI 텍스트 입력	2건	0건	2건	수정 완료 및 검수 완료

6. 미완료·이슈 및 조치 계획

ID	이슈/미완료 사항	영향도	조치 계획	완료 예정일	담당	상태
Dis-001	예외 발생 시 안내 메시지 처리	하	안내 메시지 추가	2026-09-25	정유운	완료
Dis-002	체중에 음수가 들어 갈 수 있음	중	음수가 감지되면 반영 취소	2026-09-26	정유운	완료

7. 운영 전환 계획

항목	계획 내용	담당	예정일	상태
배포	GitHub Repository를 통하여 배포 https://github.com/Ovie-hex/Software-Project-P1-Group-5	정유운	2026-09-15	완료
장애 대응	발주자가 GUI 혹은 터미널에서 발생한 오류를 사진으로 채증하여 개발자에게 보낸후 개발자는 이에 대한 시스템 보수를 실시하여 최대한 빠른 시간 이내로 GitHub에 다시 배포	정유운	2026-09-25	완료
운영 인수인계	해당 없음	-	-	-
데이터 이관	해당 없음	-	-	-
사용자 교육	대상 : 오은찬 (2023114991) 방식 : GitHub ReadMe.md 파일 및 main.py 내 설명으로 숙지 일정 : 2026-09-25	정유운	2026-09-25	완료

8. 산출물 목록

번호	산출물명	버전	파일/저장 위치	제출 여부	비고
1	소프트웨어 설계서	[v0.1 ~ v1.0]	https://github.com/Ovie-hex/Software-Project-P1-Group-5	제출	
2	기획 및 요구사항 정의서	-	https://github.com/Ovie-hex/Software-Project-P1-Group-5	제출	
3	시험 보고서	[v1 ~ v3]	https://github.com/Ovie-hex/Software-Project-P1-Group-5	제출	
4	소프트웨어 검수 보고서	[v0.1 ~ v0.3]	https://github.com/Ovie-hex/Software-Project-P1-Group-5	제출	

9. 종합 의견 및 완료 판단

처음에 생각했던 개발 기간보다 2배 정도 기간이 길어졌으나 계획에 맞게 모든 요구사항을 반영하고 개발자 기준에서 예외 처리를 어느정도 완료하였다. 이 앱을 만듦으로써 사용자가 좀 더 건강 관리에 신경 쓸 수 있게 되었으며 이후 주기적이지 않더라도 업데이트를 통하여 해당 어플리케이션을 발전시킬 예정이다. 현재까지의 잔여 위험은 검수 기준 발견되지 않기는 하나 만약에 오류가 발생하는 경우 웹메일을 통한 연락이 필요하다. 운영은 현재 개발자가 하고 있으나 필요시 운영 전환이 가능 하며 이 경우 깃허브 레포지토리 권한을 차기 운영자에게 양도한다.

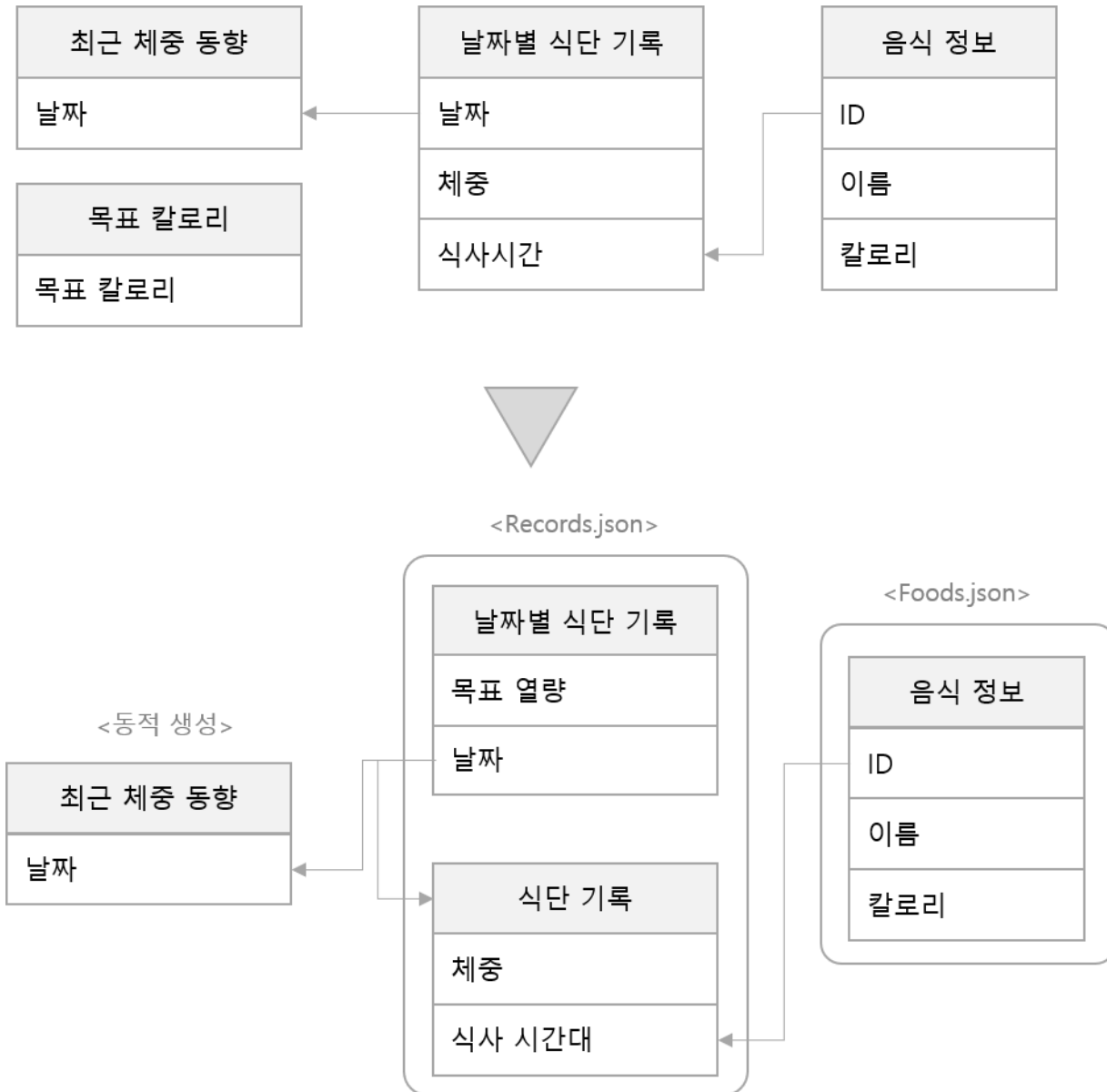
[부록]

1. 설계서 변경 사항

1. 제출물 링크 변경 (<https://github.com/Ovie-hex/Software-Project-P1-Group-5>)

↳ 사유 : 개인 계정 -> 비즈니스용 계정 으로 변경하였음

2. 데이터 파일 구조 변경



설계 원칙에 따르면 데이터 구조가 유지되어야 하나 개발할 때 필요한 부분을 구현하는데 최적의 구조가 필요했으므로 이와 같이 변경하였다. 따라서 v1.2 부터는 데이터가 호환이 되지 않으며 그 이전 버전의 경우 Record.json 부분에서 기록을 일일이 옮길 필요가 있어 불편함을 겪을 수 있다.

3. 사용되는 라이브러리 추가

(3.1) Operation_Manager

- + copy(deepcopy) : 딕셔너리의 깊은 복사 지원
- + datetime(datetime, date, timedelta) : 현재 기기의 날짜를 제공
- + math(isfinite) : 입력 값의 유효성 검사 (값이 무한이 아니도록)
- + time : 프로그램 실행 시간 반환

(3.2) Data_Manager

- + ctypes : c언어로 구성된 Windows 시스템 파일 파싱 및 사용
- + os : 운영 체제 기능 사용
- + shutil : 폴더 관리
- + contextlib : 파일 관리

2. 구현

1. Gui_Manager

- Tkinter 객체를 관리하기 위한 클래스를 정의하고 배치하는 역할을 수행한다.

(1.1) 관리 클래스

Window 클래스

- Page 객체를 관리하기 위한 클래스로 **tk.Tk** 클래스를 실 객체로 지정하여 해당 객체의 자식인 Page들을 관리한다.
- 멤버 변수 (I/O) [name, size, resizable]
- 멤버 변수 (O Only) [obj, pages]
- 멤버 함수

Window.destroy(name) : 해당 클래스의 관리 객체 및 실 객체를 삭제한다.

Window.addPage(name) : Window 객체 안에 Page를 추가한다.

Window.removePage(name) : Window 객체 안에 이름에 해당하는 Page 객체를 삭제한다.

Window.changePage(name) : 현재 페이지를 (name) 페이지로 변경한다. 이 과정에서 페이지 내 모든 Widget은 주어진 Data를 통해 모두 동기화를 실시한다.

Page 클래스

- Widget객체를 관리하기 위한 클래스로 **tk.Frame** 클래스를 실 객체로 지정하여 해당 객체의 자식인 Widget들을 관리한다.

- 멤버 변수 (I/O) [name, style]
- 멤버 변수 (O Only) [parent, obj, widgets]
- 멤버 함수

Page.hidePage() : Page의 부모 Window의 changePage()를 수행하기 위한 함수로 배치된 페이지를 뺀다.

Page.placePage() : Page의 부모 Window의 changePage()를 수행하기 위한 함수로 페이지 내 Widget 객체들을 동기화 시키고 페이지를 배치한다.

Page.addWidget(name, ...) : Page 객체 안에 Widget을 추가한다. 추가적으로 Widget의 속성이나 배치를 매개변수를 통해 정할 수 있다.

Page.removeWidget(name) : Page 객체 안에 이름에 해당하는 Widget 객체를 삭제한다.

Page.destroy() : 해당 클래스의 관리 객체 및 실 객체를 삭제한다.

Widget 클래스

- Tkinter 객체를 감싸는 관리 클래스로 Page 혹은 부모 Widget에 귀속 시킴으로써 부모 클래스의 멤버 함수로 부터의 일관성 있는 처리를 보장한다.

- 멤버 변수 (I/O) [name, style, sync, event]
- 멤버 변수 (O Only) [obj, parent, placeType, placeAttribute, **activated**, childWidgets]
- 멤버 함수

(1) 내부에서만 사용하는 함수

Widget._command() : 해당 객체가 이벤트를 발생시킬 때 이 함수를 수행한다. activated 멤버 변수를 통해 해당 함수가 끝날 때 까지 이벤트 함수가 호출되지 않아 안전하게 로직 및 데이터 처리를 실시할 수 있다.

(2) 외부 공개 함수

Widget.hideWidget() : 이미 Page를 통하여 한꺼번에 배치를 뺄 수 있지만 별개로 배치를 빼고 싶을 때 사용한다.

Widget.placeWidget() : 임의로 해당 객체를 배치한다.

Widget.addChildWidget(name, ...) : 자식 Widget 객체를 추가한다.

Widget.removeChildWidget(name) : 자식 Widget 객체를 제거한다.

Widget.changePlaceAttribute(placeType, placeAttribute) : 해당 Widget의 배치 구성을 변경한다.

Widget.syncWidget() : 해당 Widget을 sync함수를 토대로 동기화한다.

(1.2) 메인 클래스

GUI 클래스

- 위 관리 클래스를 생성하고 배치 및 관리하기 위한 클래스로 해당 클래스의 생성자 아래에 주요 코드를 정의한다.

- 단 마지막 줄에 Window.obj.mainloop()를 넣도록 하자.
- 해당 클래스가 존재함으로써 main.py는 단순히 2줄만 적으면 된다.

2. Operation_Manager

- Gui_Manager의 객체의 동기화 함수 및 이벤트 함수의 주요 호출 값을 모아 놓은 모듈이다.

3. Data_Manager

- json에 데이터 삽입 및 삭제를 수행하며 트랜잭션 단위로 진행되어 안전하게 데이터를 처리한다.

[실행화면 모음]



< 1. 메인 페이지 및 식단 관리 페이지 >



< 2. 식단 조회(2가지) 및 음식 관리 페이지 >

3. 결과 분석 및 검토

대부분 내가 원하는 대로 이루어졌지만 설계 부문에서 작은 모순하고 일관성 깨짐이 여전히 발견되고 있다는 점이고 이는 유지 보수를 더욱 어렵게 만든다는 불안함을 야기시킬 수밖에 없다 따라서 더욱 좋은 설계를 찾고 구현할 필요가 있다고 느낀다. Widget의 style의 경우 속성을 바꾸는 것에서 기존 스타일 객체가 덮어씌워지기 때문에 데이터 상으로 실 객체 데이터와 모순이 발생하여 일관성이 깨진다. 이부분은 실제 프로그램에 영향을 끼치지 않지만 추후 팀프로젝트를 진행할 때는 이 점을 고쳐 나갈 필요가 있다. 나머지 설계는 딱히 문제없어 보이나 치명적인 문제가 발생하면 원인이 무엇이고 어떻게 해결해 나가야 할 지 생각 해야 할 것 같다.

4. 자체 평가서

처음에는 Tkinter 모듈을 이용하여 개발자 스스로 GUI설계를 먼저 한 다음에 AI로 나머지 부분을 전부 처리를 하려고 했다. 왜냐하면 설계 없이는 AI가 제대로 작동할 수 없을 것이라 생각했고 설계까지 AI한테 맡겨버리면 내가 통제할 수 없다고 생각 했기에 이 것만큼은 내가 만들어야 했었다. 그래서 9월 15일에 개발을 시작했으나 말 그대로 쉽지가 않았다. 쉬는 틈 없이 매일매일 하루에 4시간 이상 설계를 한 것 같다. 스스로 생각하면서 더 좋은 설계가 있으면 기존에 있던 설계를 갈아 엮을 정도로 끈질기게 했다. 결국 9월 24일 약 9일만에 GUI 설계를 완성하면서 내가 이때까지 만든 설계 부분 중에 가장 코드 줄이 길었다고 생각했다. 완성한 설계의 이름은 바로 Cascade GUI Wrapping, GUI 객체별로 관리하는 클래스를 만들어 객체들이 따로 놀지 않고 내 통제안에 있으니

만들고 나면 개발 간에 얼마나 편했는지 모르겠다. 데이터가 바뀌면 페이지의 함수를 통하여 모든 위젯을 해당 데이터에 맞게 동기화 하는 것이 핵심이다. 솔직히 설계를 끝낼 수만 있다면 나머지는 AI가 만들어줘도 통제가 훨씬 쉬워질 것이라 생각하기에 드디어 안도하는 마음으로 본 개발을 시작하였다. 그렇게 1일도 안되어서 모든 요구사항을 탑재한 어플리케이션이 완성되었고 기쁜 마음으로 발주자에게 검수를 받았다. 나는 검수가 성공할 꺼라 생각했지만 예상치 못한 자잘한 변수로 실패 하고야 만다. 다만 해당 사항들은 모두 앱에 치명적인 피해를 주지 않았기에 나는 안심하고 여유롭게 모든 사항을 고쳐서 결국 모든 과정을 마치고 만다. 이 프로젝트를 수행하면서 느낀 점은 결국은 내가 스스로 부딪히지 않으면 성장할 수 없다는 것을 뼈저리게 느꼈다. "좋은 설계는 무엇인가?", "어떻게 해야 좋은 코드를 짤 수 있을까?" 이것을 다른 수업을 들으면서 계속 생각하고 휴대폰에 적고 그걸 집에 가서 구현하고 절망하고 다시 지우고를 반복하고 나니 어제보다 나은 나를 발견할 수 있어서 굉장히 좋았던 것 같다. 덕분에 객체 지향 프로그래밍을 어떻게 활용하는지 알았다.

5. 개발 일지

2026-09-15

- 개발 시작

2026-09-20

- 1차 GUI 설계 완료하였으나 일관성이 깨져서 실패 (그 이후로 2차 시도했으나 다시 일관성이 깨져서 실패)

2026-09-22

- 메인 페이지 추가

2026-09-24

- 3차 GUI 설계 완료, 일관성 테스트 성공 및 동기화 함수 수정

2026-09-25

- "청춘 일상 다이어트 총력전" v1.0 출시 (요구사항 모두 제작 완료)
- "청춘 일상 다이어트 총력전" v1.1 출시 (안내 메시지 출력 및 타입 오류 값 처리 완료)

2026-09-26

- "청춘 일상 다이어트 총력전" v1.2 출시 (빈 값 처리 및 체중 최소 범위 값 처리 완료)
- 검수 통과

2026-09-27

- 보안상의 이유로 GitHub 저장소 소유자 변경 (Comatine -> Ovie_hex)

2026-09-28

- "청춘 일상 다이어트 총력전" v1.3 출시 (식단 관리에서 식단 데이터가 존재할 때 삭제가 가능하도록 변경)

2026-09-29

- "청춘 일상 다이어트 총력전" v1.4 출시 (백업 데이터 최대 12개 까지 저장 가능하도록 변경)

6. 참고 문헌

[1] Tkinter 공식 API 문서 [<https://docs.python.org/3/library/tkinter.html>]

AI 활용 내역

AI 종류	사용 단계	질문 내용	응답 내용
ChatGPT-5	설계 오류 수정	질문 내용 및 응답 내용 모두 아래 링크에 첨부함 https://chatgpt.com/share/6abc5f19-033c-83e8-b1b2-59fae51d6159 https://chatgpt.com/share/6abc5f38-10f8-83ee-83da-dd99de5be256	
Codex(GPT-5)	Gui_manager 내 객체 생성 및 할당, Operation_Manager의 전반적인 기능 Data_manager의 전반적인 기능	질문 내용 및 응답 내용 모두 아래 링크에 첨부함 https://chatgpt.com/s/cx_6abb0a29290c81918056b28cb49288df	